

NESTED JSON RESPONSE VALIDATION

Senior SDET & Lead QA Interview Preparation

What this guide covers

REST Assured JSONPath assertions, nested objects and arrays, dynamic collection validation, Jackson POJO deserialization, JSON Schema validation, and an interview-ready answer.

Practical stack: Java + REST Assured + TestNG + Jackson

Interview focus: Explain how to make validations reliable, reusable and maintainable—not just how to assert one field.

Includes properly formatted JSON and Java code examples.

1. Understand the nested JSON response

Assume a customer API returns basic customer details, a nested address object, and an array of orders. This example contains both nested objects and an array of objects.

Example API response

```
{
  "id": 101,
  "name": "Swapnil",
  "email": "swapnil@gmail.com",
  "address": {
    "city": "Pune",
    "state": "Maharashtra",
    "pincode": 411045
  },
  "orders": [
    {
      "orderId": 501,
      "product": "Laptop",
      "price": 75000,
      "status": "DELIVERED"
    },
    {
      "orderId": 502,
      "product": "Mobile",
      "price": 25000,
      "status": "SHIPPED"
    }
  ]
}
```

Identify the structure

Structure	Example	Meaning
Root fields	id, name, email	Properties directly inside the response object.
Nested object	address.city	A property inside the address object.
Nested array	orders[0].product	A property inside the first object in the orders array.

Key idea: Dot notation accesses nested object properties. Square brackets access array elements by index.

2. Approach 1 — JSONPath with REST Assured

Use REST Assured's `body()` method with JSONPath expressions and Hamcrest matchers to validate individual response fields.

Field-level assertions

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

import org.testng.annotations.Test;

public class NestedJsonValidationTest {

    @Test
    public void validateCustomerResponse() {

        given()
            .baseUrl("https://api.example.com")
            .when()
            .get("/customers/101")
            .then()
            .statusCode(200)

            // Root-level fields
            .body("id", equalTo(101))
            .body("name", equalTo("Swapnil"))
            .body("email", equalTo("swapnil@gmail.com"))

            // Nested object
            .body("address.city", equalTo("Pune"))
            .body("address.state", equalTo("Maharashtra"))
            .body("address.pincode", equalTo(411045))

            // Nested array: selected elements
            .body("orders[0].orderId", equalTo(501))
            .body("orders[0].product", equalTo("Laptop"))
            .body("orders[0].status", equalTo("DELIVERED"))
            .body("orders[1].product", equalTo("Mobile"))
            .body("orders[1].status", equalTo("SHIPPED"));
    }
}
```

Common JSONPath expressions

Expression	What it accesses
<code>name</code>	Root-level name.
<code>address.city</code>	City inside address.
<code>orders[0].product</code>	Product in first order.
<code>orders[1].status</code>	Status in second order.
<code>orders.size()</code>	Number of orders (REST Assured GPath expression).

When to use: JSONPath is concise and effective for focused field-level assertions. It is often the quickest approach for straightforward API tests.

3. Validate arrays without hardcoding indexes

A common weakness is validating only `orders[0]` and `orders[1]`. If the API returns a variable number of records, use collection-level assertions for rules that apply to every element.

Validate every order

```
.then()  
  .body("orders.size()", greaterThan(0))  
  .body("orders.orderId", everyItem(notNullValue()))  
  .body("orders.product", everyItem(notNullValue()))  
  .body("orders.price", everyItem(greaterThan(0)));
```

Assertion	Purpose
<code>orders.size() > 0</code>	At least one order must exist.
<code>everyItem(notNullValue())</code>	Every order must have a non-null value for the selected field.
<code>greaterThan(0)</code>	Every order price must be positive.

Check whether a product exists anywhere in the array

```
.then()  
  .body("orders.product", hasItem("Laptop"));
```

Senior-level consideration: Validate business rules as well as field presence. For example, a DELIVERED order may require delivery details, if the API contract specifies them.

Use fixed indexes only when array ordering is guaranteed by the contract or when the test specifically checks a particular position.

4. Approach 2 — Deserialize JSON into nested POJOs

For complex responses, Jackson's `ObjectMapper` converts JSON into Java objects. This is useful when tests reuse response data, perform several business validations, or need a strongly typed response model.

Customer.java

```
import java.util.List;

public class Customer {
    private int id;
    private String name;
    private String email;
    private Address address;
    private List<Order> orders;

    // Generate getters and setters.
}
```

Address.java and Order.java

```
public class Address {
    private String city;
    private String state;
    private int pincode;

    // Generate getters and setters.
}

public class Order {
    private int orderId;
    private String product;
    private double price;
    private String status;

    // Generate getters and setters.
}
```

Test class

```

import com.fasterxml.jackson.databind.ObjectMapper;
import io.restassured.response.Response;
import org.testng.Assert;
import org.testng.annotations.Test;

import static io.restassured.RestAssured.given;

public class CustomerPojoTest {

    @Test
    public void validateNestedJsonUsingPojo() throws Exception {

        Response response =
            given()
                .baseUrl("https://api.example.com")
                .when()
                .get("/customers/101");

        Assert.assertEquals(response.getStatusCode(), 200);

        ObjectMapper mapper = new ObjectMapper();
        Customer customer =
            mapper.readValue(response.asString(), Customer.class);

        Assert.assertEquals(customer.getId(), 101);
        Assert.assertEquals(customer.getName(), "Swapnil");

        // Nested object validation
        Assert.assertEquals(
            customer.getAddress().getCity(), "Pune");

        // Nested array validation
        Assert.assertFalse(customer.getOrders().isEmpty());
        Assert.assertEquals(
            customer.getOrders().get(0).getProduct(), "Laptop");

        Assert.assertTrue(
            customer.getOrders().get(0).getPrice() > 0);
    }
}

```

Note: The POJO classes need suitable getters/setters or compatible Jackson annotations/constructors. Add null checks where fields are optional or may be absent.

5. Approach 3 — JSON Schema and contract validation

Field-level assertions verify selected values. JSON Schema validation checks the response structure, required properties, data types and constraints. Contract testing checks compatibility between API providers and consumers.

Simplified JSON Schema

```
{
  "type": "object",
  "required": ["id", "name", "address", "orders"],
  "properties": {
    "id": {
      "type": "integer"
    },
    "name": {
      "type": "string"
    },
    "address": {
      "type": "object",
      "required": ["city", "state", "pincode"],
      "properties": {
        "city": { "type": "string" },
        "state": { "type": "string" },
        "pincode": { "type": "integer" }
      }
    },
    "orders": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["orderId", "product", "price", "status"],
        "properties": {
          "orderId": { "type": "integer" },
          "product": { "type": "string" },
          "price": { "type": "number" },
          "status": { "type": "string" }
        }
      }
    }
  }
}
```

REST Assured schema assertion

```
import static io.restassured.module.json.JsonSchemaValidator
    .matchesJsonSchemaInClasspath;

given()
    .baseUrl("https://api.example.com")
    .when()
    .get("/customers/101")
    .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath("customer-schema.json"));
```

Add the REST Assured JSON Schema Validator module compatible with your REST Assured version. Put `customer-schema.json` on the test classpath, commonly `src/test/resources`.

What should a robust API test validate?

Type	Example
HTTP response	Status code, headers and content type.
Structure	Required root and nested properties exist.
Data types	ID is numeric; name is a string; orders is an array.
Values	Expected city, valid order status, positive price.
Collections	Array size, item structure and field values.

Business rules	Related fields satisfy business requirements.
Contract	Schema or consumer-provider compatibility.

6. Interview-ready answer

Use this response in a Senior SDET / Lead QA interview. Adapt the example to a system you have actually tested.

Interview-ready answer

To validate a nested JSON response, I use REST Assured with JSONPath for field-level assertions. I first validate the HTTP status code and content type, then validate root-level fields, nested objects using dot notation, and nested arrays using indexes or collection-level matchers.

For example, if a customer API returns customer details, an address object and an array of orders, I can validate fields such as `address.city` and `orders[0].orderId`.

For dynamic arrays, I avoid hardcoding indexes and use collection matchers such as `everyItem()` to validate all elements. For complex responses, I deserialize the JSON into nested POJO classes using Jackson's `ObjectMapper`. This makes the framework more maintainable and allows reusable business-level assertions.

I also use JSON Schema validation to verify the response structure, required fields and data types against the API contract. Finally, I include business-rule validations, because a response can be structurally correct but still contain incorrect business data.

How to choose the right approach

Approach	Best used when	Strength
JSONPath	A few direct field assertions.	Concise and quick to write.
POJO + Jackson	Complex responses and reusable test logic.	Typed objects and maintainability.
JSON Schema	Checking structure and data types.	Broad structural checks.

Likely follow-up questions

Q: What if the array contains 100 records?

Use collection-level assertions or extract the list and iterate through it. Avoid validating only a few fixed indexes unless order is part of the contract.

Q: What if a nested field is optional?

Follow the API contract. A property may be absent, explicitly null, or required; these cases are not interchangeable.

Q: Is schema validation enough?

No. Schema validation checks structure and types, but not whether business values are correct. Combine it with value and business-rule assertions.

Q: When would you use POJO deserialization?

When the response is complex or reused across validations, and typed Java objects make the tests clearer.

7. Common mistakes and final checklist

Avoid these mistakes

- **Hardcoding array indexes everywhere:** Tests can become brittle when array size or ordering changes.
- **Checking only status code:** HTTP 200 does not guarantee correct response data.
- **Confusing missing and null:** A missing property and a property with a null value may have different contract meanings.
- **Ignoring data types:** "101" is a string; 101 is a number.
- **Only validating schema:** A schema-valid response can still violate business rules.
- **Over-validating unstable fields:** Avoid asserting generated IDs, timestamps or ordering unless requirements make them deterministic.

Framework checklist

- Validate status code and content type.
- Validate required root-level properties.
- Validate nested objects and required fields.
- Validate arrays for size, item structure and values.
- Validate data types, nullability and optional fields.
- Validate business rules across related fields.
- Apply schema or contract validation where appropriate.
- Keep reusable assertions in a common validation layer.
- Produce clear failure messages identifying the failing JSON path.

Final takeaway

For a Senior SDET answer, show that you can combine JSONPath, POJO deserialization and schema validation rather than relying on one technique. Choose the simplest approach that provides reliable, maintainable coverage for the API contract and business behaviour.

Reference documentation

REST Assured: <https://rest-assured.io/>

Jackson: <https://github.com/FasterXML/jackson>

JSON Schema: <https://json-schema.org/>